Reporte Técnico de Actividades Práctico-Experimentales Nro. 002

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Maria Elizabeth Chuico Medina
Asignatura	Sistemas Distribuidos
Ciclo	6 A
Unidad	1
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	002
Título de la Práctica	Arquitectura de Sistemas Distribuidos (comunicación cliente–servidor con archivos)
Nombre del Docente	Ing. José O. Guamán Q.
Fecha	06 de octubre de 2026 09 de octubre de 2026
Horario	Lunes 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica

Escriba el objetivo de la práctica (copiado de la guía proporcionada por el docente).

Comprender la arquitectura de sistemas distribuidos mediante el desarrollo independiente de sus componentes (cliente y servidor).

3. Materiales y reactivos

Guia practica

Se requiere un sistema que permita crear un servidor y un cliente con un canal de comunicación (archivo de texto), se debe evidenciar que se esta llenado el archivo

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación
- IDE de programación
- Archivo de texto

5. Procedimiento / Metodología Ejecutada

Describa brevemente los pasos que siguió durante la ejecución de la práctica.

El desarrollo de la practica se desarrollo en tres partes como se indicaba en la guía, el servidor y cliente se desarrollan como proyectos por separado sin que ninguno conozca el código de otro.

5.1 Preparacion: estructura del proyecto

- Se creo una carpeta de l proyecto denominda APE2 con tres subcarpetas Servidor, cliente y comunicación. Los archivos de comunicación están separados de cliente y servidor.
- Se determino el contrato de mensajes conteniendo el id y texto.

```
APE2/
├── servidor/
│   ├── servidor.py             ← Parte 1
│   └── estado_servidor.txt     ← IDs ya procesados (lo crea el servidor)
├── cliente/
│   └── cliente.py              ← Parte 2
└── comunicacion/               ← canal compartido, fuera de ambos proyectos
    ├── entrada_servidor.txt    ← el cliente escribe:   id|mensaje
    └── salida_servidor.txt     ← el servidor escribe:  id|resultado
```

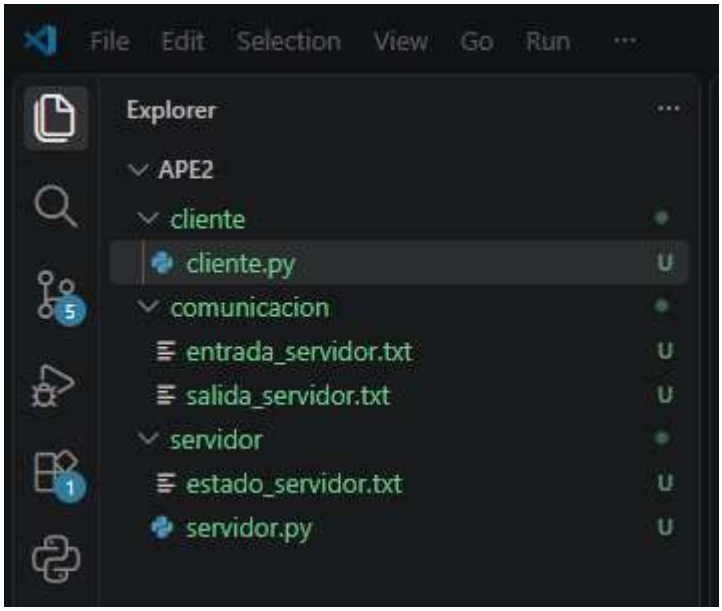


Figura 1. *Estructura del proyecto APE2 en Visual Studio Code: carpetas del cliente, del servidor y del canal de comunicación.*

5.2 PARTE1: Desarrollo del servidor(servidor.py)

- **Creación y funcionamiento del servidor:** se creó el archivo *servidor.py* junto con *entrada_servidor.txt*. Si los archivos de comunicación no existen

el servidor los crea al ejecutar. Luego se programó un bucle que revisa el archivo de entrada cada segundo como si fuera un servicio siempre activo. Para saber que los mensajes son nuevos, el servidor recuerda hasta donde leyó la ultima vez por lo que cada mensaje nuevo se convierte a mayúsculas y se invierte y la respuesta se escribe en *salida_servidor.txt* con el formato *id | resultado*.

- **Control de mensajes repetidos y archivos vacíos:** para evitar no procesar dos veces el mismo mensaje, el servidor guarda en *estado_servidor.txt* los IDs que ya pasaron, así no repite respuestas aunque se reinicie. Si el archivo de entrada está vacío o todavía no existe, el servidor espera a la siguiente revisión. También ignora las líneas en blanco o las que no tienen el separador |.

5.3 PARTE2: Desarrollo del cliente(cliente.py)

- **Creación y envío de mensajes desde el cliente:** se creo el archivo *cliente.py* que le pide un mensaje al usuario con *input()* a cada mensaje le asigna un ID único y lo escribe al final de *entrada_servidor.txt* con el formato *id | mensaje*. Después espera 2 segundos para simular la latencia de la red.

- **Recepción de la respuesta:** después de la espera el cliente busca en *salida_servidor.txt* la línea con su ID. Si no aparece vuelve a revisar cada medio segundo hasta un 10s. Cuando la encuentra muestra el resultado en pantalla en caso de no llegar avisa que el servidor tal vez no esta en ejecución. El cliente no necesita saber como trabaja el servidor por dentro: solo conoce las rutas de los dos archivos y el formato *id | texto*.

5.4 PARTE3: Integracion del sistema

- **Puesta en marcha del sistema:** se abrieron dos terminales en Visual Studio Code. En la primera se ejecutó el servidor (python servidor.py dentro de servidor/) y ya con el servidor activo, en la segunda se ejecuto el cliente (python cliente.py "mensaje" dentro de cliente/).

- **Envío de mensajes y verificación:** se enviaron 4 mensajes después de cada uno se revisaron *entrada_servidor.txt* y *salida_servidor.txt* con el comando type para ver cómo se iban llenando. Así se comprobó que cada mensaje hacía el recorrido completo (cliente ->entrada -> servidor -> salida -> cliente) y que todas las respuestas eran correctas.

6 Resultados

5.1 Mensajes procesados

Se enviaron **4 mensajes** todos se procesaron correctamente fueron y cada respuesta llego unos 2s después del envío:

#	ID	Mensaje enviado (cliente)	Respuesta recibida (servidor)
1	d0936dea	Hola mundo	ODNUM ALOH
2	f3852ab8	Sistemas Distribuidos	SODIUBIRTSID SAMETSIS
3	2a2c4d9a	Universidad Nacional de Loja	AJOL ED LANOICAN DADISREVINU
4	de6bf232	cliente servidor con archivos	SOVIHCRA NOC RODIVRES ETNEILC

5.2 Evidencias

- ***entrada_servidor.txt siendo llenado por el cliente***

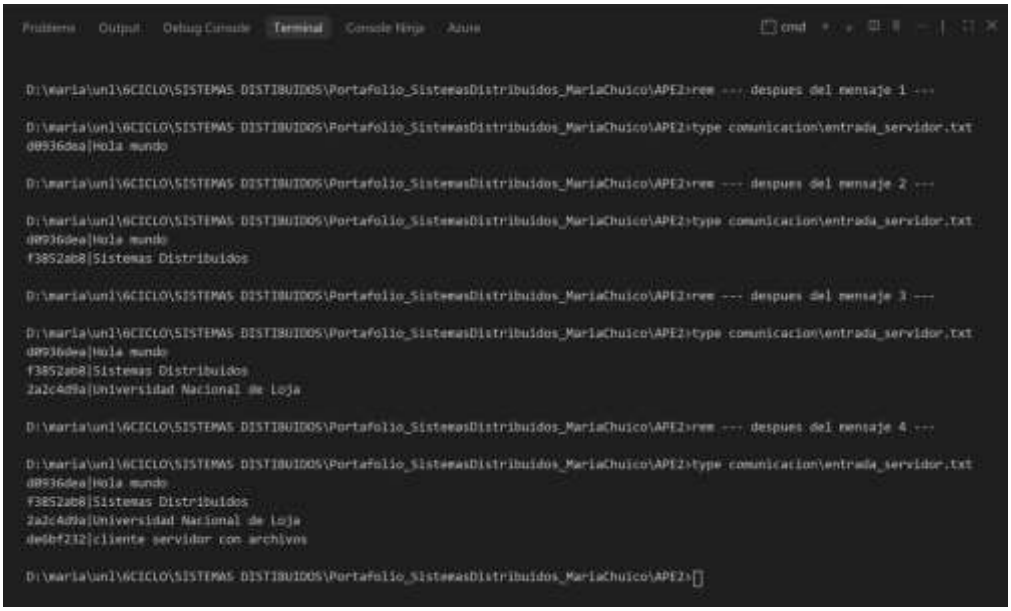


Figura 2. *Contenido de entrada_servidor.txt después de cada uno de los 4 mensajes.*

- ***salida_servidor.txt con el resultado procesado***

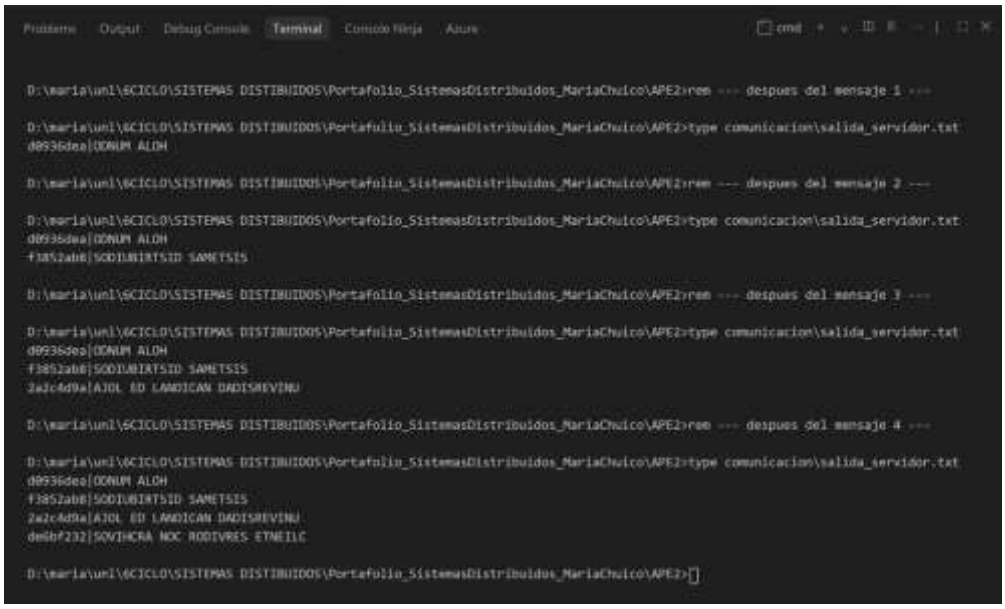


Figura 3. *Contenido de salida_servidor.txt con las respuestas del servidor.*

- ***Al menos 3 mensajes procesados correctamente***



Figura 4. *Consola del servidor procesando los 4 mensajes.*

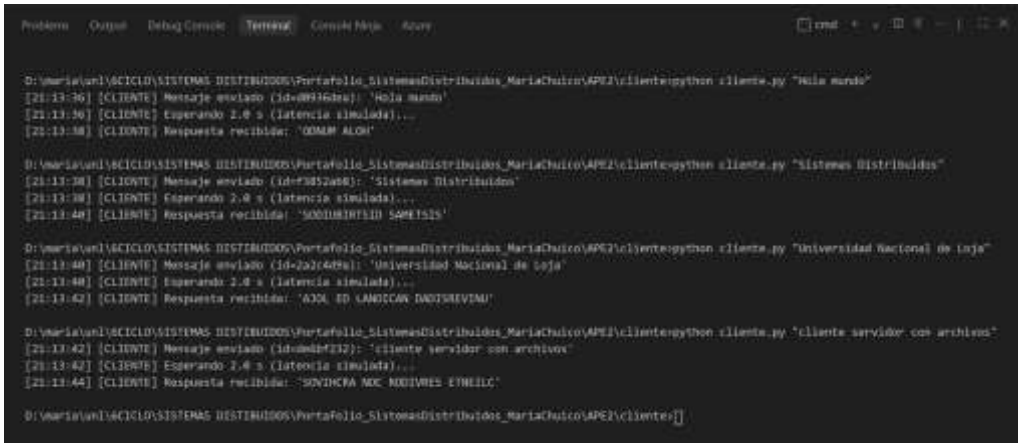


Figura 5. *Consola del cliente enviando mensajes y recibiendo las respuestas.*

5.3 Pruebas adicionales

Prueba	Resultado	Evidencia
Iniciar el servidor con los archivos vacíos	El servidor arranca sin errores y espera mensajes.	Figura 6

Prueba	Resultado	Evidencia
Ejecutar el cliente con el servidor apagado	El mensaje queda en la entrada y, tras 10 s, el cliente muestra "No se recibió respuesta. ¿El servidor está en ejecución?".	Figura 7
Reiniciar el servidor con mensajes ya procesados	Carga los 4 IDs procesados y **no genera respuestas duplicadas**.	Figura 8



Figura 6. *Archivos de comunicación vacíos (0 bytes) al iniciar el servidor.*



Figura 7. *Cliente ejecutado con el servidor apagado: aviso por tiempo de espera agotado.*



Figura 8. *Reinicio del servidor: carga 4 mensajes procesados y no genera respuestas duplicadas.*

7 Preguntas de Control

Responda a las preguntas del docente.

- **¿Qué significa que cliente y servidor estén desacoplados?**
  Significa que ninguno de los dos depende del otro, en el caso de la practica el cliente no sabe el código del servidor ni que hace nada y asi mismo el servidor no sabe quien escribe, lo único que saben es la ubicación de los archivos y que formato tiene (contrato). El desacoplamiento es temporal ya que el cliente puede dejar un mensaje aunque el servidor este apagado y cuando se encienda este lo procesa.

- **¿Qué ventajas tiene desarrollar componentes por separado?**
  - Entornos separados que dos o mas personas pueden trabajar al mismo tiempo.
  - Se puede realizar mantenibilidad ya que se puede cambiar la lógica sin tocar a cliente.
  - Se puede hacer pruebas independientes ya que cada componente se puede hacer pruebas por su lado.
  - Se puede reutilizar y ser escalables como el servidor puede atender a varios clientes y cada modulo puede ejecutarse en otra computadora
  - La tolerancia a fallos, si un modulo falla no afecta a los demás.

- **¿Qué problemas pueden surgir al integrar sistemas independientes?**
  Los problemas que pueden surgir son que no tenga un mismo formato o sintaxis en caso de la practica por ejemplo los separadores, otro inconveniente es rutas o nombres de archivos diferentes, la sincronización y tiempos, tambien un manejo de errores de diferente manera y cambios de contrato.

- **¿Qué sucede si el servidor no está ejecutándose? ¿Cómo se simula la comunicación en esta práctica?**
  Pasa que el cliente escribe el mensaje en entrada_servido.txt sin ningún problema pero no hay quien procese, tras la latencia y el tiempo de espera de 10s el cliente va ha mostrar que no hay respuesta, pero no se pierde sino que queda guardado para cuando el servidor se encienda lo procesa.
  La simulacion de la comunicación se da con un medio compartido con dos archivos de texto en un carpeta en común en el caso de la actividad `entrada_servidor.txt` cumple el papel del canal de petición (request) y `salida_servidor.txt` el del canal de respuesta (response) la espera de 2s del cliente simula la latencia y la revisión del servidor simula que el servicio si escucha las peticiones, por lo que es una comunicación indirecta y asíncrona.

- **¿Qué limitaciones tiene este modelo frente a uno real?**
  - **No es tiempo real:** el sondeo (polling) cada 1 s añade retardo y gasta CPU aunque no haya mensajes.
  - **Conflictos de escritura:** si varios procesos escriben a la vez sin bloqueos, las líneas pueden mezclarse.
  - **Dependencia del medio:** cliente y servidor deben ver el mismo sistema de archivos; si el disco falla, se cae todo.
  - **Crecimiento sin límite:** los archivos nunca se limpian y la búsqueda de respuestas es lineal.
  - **Sin seguridad ni confirmación de entrega:** cualquiera con acceso a la carpeta puede alterar los mensajes, y el cliente no sabe si su mensaje fue recibido.
  - Un sistema real usa sockets TCP/UDP, HTTP, RPC o colas de mensajes (RabbitMQ, Kafka), que ofrecen notificación inmediata, control de concurrencia y comunicación entre máquinas distintas.

- **¿Cómo se podría mejorar la sincronización?**
  - **Bloqueo de archivos** (`msvcrt.locking`, `fcntl` o la librería `filelock`) para que solo un proceso escriba a la vez.

- **Escritura atómica:** escribir en un archivo temporal y renombrarlo, para que nadie lea datos a medias.
- **Notificaciones del sistema operativo** (`watchdog`, `inotify`) en lugar de sondeo.
- **Un archivo de respuesta por petición** (`respuesta_<id>.txt`) o archivos de bandera que indiquen "listo".
- **Confirmaciones (ACK) y reintentos** con tiempo de espera.
- Como mejora de fondo, reemplazar los archivos por sockets o una cola de mensajes.

- **¿Qué pasaría si múltiples clientes usan el sistema?**

  Con esta implementación funciona en la mayoría de los casos: cada cliente usa un `id` único y agrega su línea, así que cada uno encuentra su respuesta, y el servidor atiende los mensajes en orden de llegada (FIFO). Sin embargo, aparecen riesgos:

  - Condiciones de carrera: dos clientes que escriben al mismo tiempo podrían mezclar sus líneas, porque no hay bloqueo.
  - Cuello de botella: un solo servidor atiende las peticiones una por una.
  - Contención del archivo: muchas lecturas simultáneas de `salida_servidor.txt`, que además crece rápido.

  Sin el identificador único, los clientes se robarían las respuestas entre sí. Por eso, en sistemas distribuidos, correlacionar peticiones y respuestas es indispensable.

8 Conclusiones

1. Se cumplió el objetivo de la práctica, se desarrollaron por separado el servidor y el cliente, y la única dependencia entre ellos es el contrato de archivos (rutas y formato `id|texto`). Esto demuestra el desacoplamiento, base de la arquitectura cliente–servidor.
2. La comunicación indirecta por un medio compartido funcionó: los 4 mensajes se procesaron correctamente, y el modelo tolera que el servidor esté apagado temporalmente porque los mensajes quedan guardados.
3. El modelo tiene limitaciones claras: el sondeo introduce retardo, no hay control de escritura concurrente y todo depende del sistema de archivos. Esto justifica usar sockets o colas de mensajes en sistemas reales.
4. El error de `\r\n` en Windows mostró la heterogeneidad de los sistemas distribuidos: integrar componentes exige acordar también la representación de los datos (codificación y fin de línea), no solo su significado.

9 Recomendaciones

- Definir y documentar el contrato de comunicación (rutas, formato, codificación UTF-8 y fin de línea) antes de programar cualquiera de los componentes.

- Leer y escribir los archivos en modo binario o con `newline` explícito para evitar diferencias entre Windows y Linux.
- Agregar bloqueo de archivos si se prueba el sistema con varios clientes a la vez.
- Limpiar o rotar periódicamente los archivos de comunicación para que no crezcan sin límite.
- En una siguiente práctica, reemplazar el medio compartido por sockets o una cola de mensajes y comparar tiempos de respuesta con este modelo.

10 Bibliografía / Referencias

Semana 2. Guaman, Jose. 2026

Declaración de uso de Inteligencia Artificial generativa

- **Claude Code (Anthropic), extensión de Visual Studio Code, modelo [Sonnet 5.5 / Opus 5.5].** La usé para generar la versión inicial de servidor.py y cliente.py a partir de los requisitos de la guía. También me ayudó a diagnosticar el error de lectura causado por los saltos de línea \r\n en Windows.

Prompts utilizados

Prompt 1 (servidor):

Crea servidor.py en Python 3, solo con librería estándar. Contrato: leo de ../canal/entrada_servidor.txt líneas con formato ID|mensaje y escribo en ../canal/salida_servidor.txt líneas con formato ID|resultado. Requisitos: ejecutarse continuamente (bucle con time.sleep de 1 s), detectar contenido nuevo, no reprocesar mensajes, manejar archivos vacíos o inexistentes, convertir el mensaje a mayúsculas e invertirlo, ignorar líneas mal formadas y mostrar un log con la hora.

Prompt 2 (cliente):

Crea cliente.py en Python 3, solo con librería estándar. No conozco cómo funciona el servidor; solo conozco este contrato: escribo en ../canal/entrada_servidor.txt una línea ID|mensaje y el servidor responde en ../canal/salida_servidor.txt con ID|resultado. Requisitos: pedir mensajes al usuario, generar un ID único, esperar 2 s para simular latencia, buscar la respuesta con mi ID con un tiempo máximo de espera, avisar si el servidor no responde y manejar archivos inexistentes.

Trabajo realizado por la estudiante

Ejecución y comprobacion del sistema en dos terminales y envie los mensajes de prueba. Hice las pruebas adicionales con archivos vacíos con el servidor apagado y con el reinicio del servidor y capturé todas las evidencias. Verifiqué que el código

cumpliera cada condición de la guía revisé y ajusté el contenido del informe y subí el código al repositorio Git.

Declaro que revisé, comprendí y verifiqué todo el contenido generado con apoyo de IA y asumo la responsabilidad sobre el trabajo presentado.

11 Anexos

Anexo A. Repositorio del código fuente

Link:
https://github.com/mariaechm/Portafolio_SistemasDistribuidos_MariaChuico/tree/main/APE2

Anexo B. Protocolo de mensajes

Archivo	Quién escribe	Formato	Ejemplo
`entrada_servidor.txt`	Cliente	`id|mensaje`	`d0936dea|Hola mundo`
`salida_servidor.txt`	Servidor	`id|resultado`	`d0936dea|ODNUM ALOH`

Anexo C. Fragmento clave del servidor

```
58  def procesar_mensaje(mensaje):
59      """Lógica de negocio del servidor: mayúsculas + texto invertido."""
60      return mensaje.upper()[::-1]
```

```
117                 id_msg, mensaje = linea.split("|", 1)
118                 if id_msg in procesados:
119                     continue  # NO reprocesar el mismo mensaje
120
121                 resultado = procesar_mensaje(mensaje)
122                 with open(ARCHIVO_SALIDA, "a", encoding="utf-8") as f:
123                     f.write(f"{id_msg}|{resultado}\n")
```

Figura 10. *Fragmento de `servidor.py`: procesamiento del mensaje y escritura de la respuesta sin reprocesar mensajes.*